

CLOUDERA AI | SE ENABLEMENT ASSET

MLOps on Cloudera AI

Demo Runbook — Credit Risk Model Lifecycle

By: Ravi .(SE India)

A step-by-step guide for Solutions Engineers to deliver a repeatable, end-to-end MLOps demonstration on Cloudera AI — from data generation and experiment tracking through governed model deployment, endpoint testing, retraining and CI/CD.

Field	Value
Asset type	Reusable SE demo runbook
Demo environment	CDP Public Cloud — Demo
Cloudera AI Workbench	go01-aws-workspace
Project name	MLOPS Workshop
Use case	Credit risk / loan default prediction (BFSI)
Model	XGBoost binary classifier
Demo duration	35–45 minutes (core flow ~30 min)
Audience	Data science, ML engineering, platform and risk-analytics teams
Version	1.0

Contents

Section	What it covers
1. Purpose and Scope	Who this is for, what is in and out of scope
2. What This Demo Proves	Lifecycle-to-capability mapping and the three messages to land
3. Demo Environment Reference	Environment, workbench, project URL, and the project file map
4. Prerequisites and Access	Access you need, role levels, shared-environment discipline
5. Pre-flight Checklist	T-24 hours and T-15 minutes checks
6. Demo Flow at a Glance	Nine stages with timings and key messages
7. Step-by-Step Walkthrough	Stages 0–8 with commands, expected output and talk track
8. Compressed Version	The 10-minute cut for booths and shortened slots
9. Optional Extensions	AI Registry, AI Inference, API v2, drift monitoring, Applications
10. Reset and Etiquette	Leaving the shared project ready for the next SE
11. Questions You Will Be Asked	Objection handling with honest answers
Appendix A	Features and model configuration
Appendix B	Sample payloads and response shape
Appendix C	Claim discipline — safe to claim, do not claim

1. Purpose and Scope

This runbook lets any Solutions Engineer run a credible, end-to-end MLOps demonstration on Cloudera AI without building anything first. The project is already provisioned and working in the shared demo environment; this document tells you what to click, what to say, what will appear on screen, and what to do when it does not.

1.1 Who this is for

- SEs demonstrating Cloudera AI to data science and ML engineering teams.
- SEs in BFSI, Oil & Gas and enterprise accounts where a regulated, auditable model lifecycle is the buying criterion.
- SEs who need a working reference implementation to adapt for a customer-specific PoC.

1.2 What is in scope

The demo covers the predictive ML (non-GenAI) lifecycle inside Cloudera AI Workbench: Projects, Sessions, Jobs, Experiments (MLflow), Models, and an external CI/CD gate. Cloudera AI Registry and Cloudera AI Inference are covered as optional extensions in Section 9 — they are the production-grade path and are worth naming in the conversation even when you do not click through them.

Scope note: This is a demo asset, not a production reference architecture. The dataset is synthetic and generated in-project, so there is no dependency on Data Lake, Iceberg or external connections. That keeps the demo fast and reliable, and it is also the first thing a technical audience will ask about — see the objection handling in Section 11.

2. What This Demo Proves

Anchor the demo on the lifecycle, not the model. The XGBoost classifier is deliberately simple; the story is that every stage of the lifecycle happens on one governed platform, against one copy of the data, under one security model.

Lifecycle stage	What the customer sees	Cloudera capability
Development	Interactive session, project files, chosen runtime and resource profile	Cloudera AI Workbench — Projects and Sessions
Training	Repeatable training run producing versioned artifacts	Sessions and Jobs on ML Runtimes
Experiment tracking	Every run's parameters, metrics and artifacts captured automatically	Experiments (native MLflow integration)
Model comparison	Side-by-side run comparison, charts, best-run selection	Experiments — Run Comparison view
Quality gate	Deployment blocked when ROC-AUC or F1 falls below threshold	Validation Job with non-zero exit code
Deployment	Authenticated, always-on REST endpoint with replicas	Models — Model Deployment
Consumption	Live scoring from the UI and from curl	Model endpoint with access key
Retraining	New training run, rebuild, zero-downtime redeploy	Jobs plus Model rebuild

Lifecycle stage	What the customer sees	Cloudera capability
Monitoring	Predictions and outcomes recorded, decay made measurable	Model metrics store plus ground-truth tracking
Automation	Push to Git triggers retrain and KPI validation	External CI/CD plus Cloudera AI API v2
Governance	Versioned, lineage-tracked model as a managed asset	Cloudera AI Registry (Section 9)

2.1 The three sentences to land

- Every experiment is tracked automatically — you can always answer "which run produced the model in production?"
- A model cannot reach an endpoint without passing an explicit, code-defined KPI gate.
- The whole lifecycle runs where the data already is, under SDX security and governance — no export, no separate ML silo.

3. Demo Environment Reference

Item	Value
Environment	CDP Public Cloud — Demo
Cloudera AI Workbench	go01-aws-workspace
Workbench URL	https://ml-dbf64d1-783.go01-dem.ylcu-atmi.cloudera.site
Project name	MLOPS Workshop
Project URL	https://ml-dbf64d1-783.go01-dem.ylcu-atmi.cloudera.site/rsingh/mlops-workshop
Session runtime	Python 3.10 — Standard (JupyterLab or Workbench editor)
Minimum resource profile	2 vCPU / 4 GiB (4 vCPU / 8 GiB recommended)
Model deployment size	2 vCPU / 4 GiB, 1 replica
Model deployment name	Workshop - Ravi MLOPS
MLflow experiment name	credit-risk-model

Direct project link:

<https://ml-dbf64d1-783.go01-dem.ylcu-atmi.cloudera.site/rsingh/mlops-workshop>

3.1 Project file map

Everything below is visible in the project's Files tab. Scripts are numbered in execution order.

File	Role in the demo	Produces
01_generate_data.py	Generates a 10,000-row synthetic loan dataset with realistic risk drivers	loan_data.csv
02_train_model.py	Trains the XGBoost classifier and logs parameters, metrics and artifacts to MLflow	credit_risk_model.pkl, label_encoder.pkl, one Experiment run

File	Role in the demo	Produces
03_predict.py	Serves the model as a Flask/Gunicorn API inside a session (development-path serving)	Local REST API on the session port
04_test_api.py	Fires a low-risk and a high-risk request at the session API	Console output with probabilities
05_validate_model.py	KPI gate — exits non-zero if ROC-AUC or F1 misses threshold	Pass/fail with exit code
cml_model.py	Model Deployment entry point; exposes predict(args) for the managed endpoint	Endpoint response JSON
cds-sw-build.sh	Build script — installs dependencies, then generates data and trains so artifacts are baked into the model image	Deployable image
requirements.txt	Pinned dependencies (scikit-learn 1.3.0, XGBoost 1.7.6, numpy < 2.0)	—

Why mlflow is not in requirements.txt: Cloudera AI pre-installs MLflow in every session through the mlflow-cml-plugin and pins the version. Installing a different MLflow on top breaks the plugin and the Experiments integration. If a customer asks why the file looks incomplete, this is a good, credible answer — and a real platform detail worth knowing.

4. Prerequisites and Access

4.1 Access you need before the call

Requirement	How to confirm	If missing
CDP Public Cloud Demo tenant access	You can sign in at the workbench URL with your Cloudera SSO	Request access through the internal demo environment process
Access to workbench go01-aws-workspace	The workbench appears in Cloudera AI > Workbenches and opens	Ask the demo environment admin to grant workbench access
Collaborator access on MLOPS Workshop	The project opens from the project URL and you can start a session	Ask the project owner to add you under Project Settings > Collaborators
Sufficient workbench quota	A session starts within about a minute	Stop idle sessions, or reduce the resource profile
API key (optional)	User Settings > API Keys shows a legacy API key	Only needed for the API v2 / CI-CD extension in Section 9.3

4.2 Roles: what your access level allows

- **Viewer** — can watch runs and read files but cannot start sessions or deploy. Not sufficient to present.
- **Contributor** — can run sessions, jobs and deploy models. This is the level you want.
- **Admin / Owner** — additionally manages collaborators, environment variables and deletion. Only needed for reset (Section 10).

Shared environment discipline: This project is shared across the SE team. Do not delete the deployed model, do not rename files, and do not commit changes to the project's Git remote. If you need to experiment, create your own project from the same Git remote (Project Settings > Git shows the remote) and demo from that copy.

5. Pre-flight Checklist

Run through this before every customer session. It takes about ten minutes and prevents almost every way this demo fails in front of a customer.

5.1 T-24 hours

#	Check	Expected state
1	Sign in to the workbench and open the project	Project loads, Files tab lists all eight files
2	Confirm the workbench is running, not suspended	Status shows Running in Cloudera AI > Workbenches
3	Open Model Deployments and check "Workshop - Ravi MLOPS"	Status is Running. It is often left Stopped — press Start and allow time for it to come up
4	Open Experiments > credit-risk-model and check the run history	Two or three runs exist AT DIFFERENT hyperparameters. Runs with identical params make the comparison view worthless
5	Confirm the deployed model still returns a prediction from the Test tab	success: true with a probability and a risk label

5.2 T-15 minutes

#	Check	Expected state
1	Start a session (Python 3.10, Workbench editor, 2 vCPU / 4 GiB)	Session reaches Running; terminal is available
2	Run the dependency install once so it is warm	pip completes without resolver errors
3	Confirm loan_data.csv and the two .pkl files are present, or plan to generate them live	Files tab shows them, or you will run Stages 2 and 3
4	Confirm the deployment reports 1/1 replicas, not 0/1	A Stopped deployment cannot serve Stage 6 — start it well before the call
5	Have the model access key copied for the curl step	Model Deployments > Workshop - Ravi MLOPS > Settings
6	Close unrelated tabs and clear the terminal	Clean screen share

Time-saver: Pre-generating loan_data.csv and the .pkl artifacts before the call cuts about four minutes of waiting. Generate live only if the audience is technical enough to care about determinism (numpy seed 42, random_state 42 — the run is reproducible, which is itself a talking point).

6. Demo Flow at a Glance

Stage	What you do	Time	Key message
0	Open the project, start a session	3 min	One governed workspace per use case
1	Generate the dataset	2 min	Reproducible, versioned inputs
2	Train the model	5 min	Training is a first-class, repeatable workload
3	Review runs in Experiments	5 min	Every run is tracked automatically — traceability
4	Run the KPI validation gate	3 min	Quality is enforced in code, not by convention
5	Deploy the model endpoint	6 min	Managed, authenticated, always-on serving
6	Test the endpoint (UI and curl)	4 min	Any application can consume it
7	Retrain and rebuild	4 min	Zero-downtime model refresh
8	CI/CD and governance wrap-up	4 min	The lifecycle is automatable end to end

If you have only 20 minutes, run Stages 0, 3, 5, 6 and 8. Skip live training and use the historical Experiment runs — the comparison view is the strongest single screen in the demo.

7. Step-by-Step Walkthrough

Stage 0 — Open the Project and Start a Session

Steps

1. Open the project directly: <https://ml-dbf6c64d1-783.go01-dem.ylcl-atmi.cloudera.site/rsingh/mlops-workshop>
2. Land on the Overview tab. Point out the left navigation: Overview, AI Studios, Sessions, Data, Experiments, Model Deployments, Jobs, Applications, Files, Collaborators, Project Settings.
3. Open Files and walk the numbered scripts top to bottom — this is the lifecycle in file form.
4. Click New Session and configure: Editor = Workbench, Kernel = Python 3.10, Edition = Standard, Resource Profile = 2 vCPU / 4 GiB. Click Start Session.
5. When the session is Running, open the **Terminal** tab and install dependencies:

```
pip install -r requirements.txt
```

Talk track

"A project in Cloudera AI is an isolated workspace — code, files, libraries, environment variables and access control, scoped to one use case. When I start a session I choose the runtime and the compute independently of the code. Nothing was installed on my laptop, and no data left the platform. That distinction matters when your data is governed."

What to watch for: Session start normally takes 30–60 seconds. If it sits in Pending for more than two minutes, the workbench is out of quota — stop idle sessions or drop to a smaller profile. Do not narrate the wait; use it to walk the Files tab.

Stage 1 — Generate the Dataset

Steps

1. In the session terminal, run:

```
python 01_generate_data.py
```

2. Confirm the output line reporting row count and default rate.
3. Open the Files tab and show that loan_data.csv now exists in the project filesystem.

Expected output

```
Dataset generated: 10000 rows, default rate: 39.00%
```

What the data represents

Eight features drive a loan default label: loan amount, annual income, credit score, employment years, debt-to-income ratio, number of open credit lines, number of past delinquencies, and loan purpose. The default label is derived from a weighted risk score, so credit score, debt-to-income and delinquency history carry the strongest signal — which is what a risk analyst would expect.

Talk track

"I'm generating synthetic data so this demo is self-contained and reproducible — same seed, same dataset, every time. In your environment this step would be a Spark or Hive read against curated tables in the Data Lake, or an Iceberg table through a Data Connection. The rest of the lifecycle is identical."

Stage 2 — Train the Model with Experiment Tracking

Steps

1. In the terminal, run:

```
python 02_train_model.py
```

2. Read out the classification report and the ROC-AUC line as they appear.
3. Note the MLflow run_id printed at the end — you will use it to find the run in the Experiments UI.
4. Confirm credit_risk_model.pkl and label_encoder.pkl now exist in the Files tab.

What gets logged automatically

Category	Logged values
Hyperparameters	n_estimators, max_depth, learning_rate, eval_metric, random_state
Dataset statistics	test_size, train_samples, test_samples, default_rate
Headline KPIs	roc_auc, accuracy
Default class (class 1)	precision_default, recall_default, f1_default
No-default class (class 0)	precision_no_default, recall_no_default, f1_no_default
Artifacts	XGBoost model with signature and input example, plus label_encoder.pkl

The detail that lands with engineers: The script contains no MLflow tracking URI, no server address and no credentials. Cloudera AI sets MLFLOW_TRACKING_URI in every session automatically, so the same code runs locally, in a session, in a Job, or in a CI pipeline without modification. Say this out loud — it is the difference between an integration and a bolt-on.

Talk track

"Training here is just a script — but because it runs inside Cloudera AI, the run is captured. Parameters, metrics, the serialised model, the encoder, the input signature. Nobody had to remember to record anything, and nobody can quietly not record it. In six months, when a regulator asks which run produced the model that declined an application, that question has an answer."

Stage 3 — Compare Runs in Experiments

This is the strongest screen in the demo. Do not rush it.

Steps

1. Click Experiments in the project's left navigation.
2. Open the credit-risk-model experiment. The run list shows every training run with its parameters and metrics as sortable columns.
3. Sort by roc_auc to identify the best run.
4. Select two or three runs using the checkboxes and click Compare.
5. In the comparison view, show the parameter and metric diff, then switch to a chart — parallel coordinates or scatter — to show how a hyperparameter moves the KPI.
6. Open a single run and show the Artifacts section: the model, the signature, the input example and the encoder are all attached to the run.

If there is only one run: The comparison view needs at least two. Re-run `02_train_model.py` once or twice with a changed `max_depth` or `learning_rate` before the call, or use runs left by previous SEs. A single-run Experiments screen undersells the capability badly.

Talk track

"This is standard MLflow — open source, no lock-in — but it is running as a managed service inside the platform, wired to project permissions. Your data scientists keep the API they already know. Your platform team gets a tracking server they did not have to deploy, secure, back up or patch."

Likely question: is this really just MLflow?

Yes, and say so plainly. The Cloudera value is that it is integrated, secured under SDX, tied to project-level access control, and connected onward to AI Registry and deployment — not that the tracking API is proprietary. Claiming otherwise will cost you credibility with anyone who has used MLflow.

Stage 4 — Enforce the KPI Gate

Steps

1. In the terminal, run:

```
python 05_validate_model.py
```

2. Show the classification report, then the two threshold lines, then the pass/fail verdict.
3. Open `05_validate_model.py` in the editor and show the thresholds at the top of the file.

Thresholds as configured

```
ROC_AUC_MIN      = 0.72
F1_DEFAULT_MIN = 0.50  # F1 for class 1 (default) — the business-critical class
```

The script evaluates on the held-out test set using the same split and `random_state` as training, then exits with code 0 on pass and code 1 on fail. That exit code is what makes it a gate: as a Job it turns red, and in a CI pipeline it blocks the build.

Why F1 on the default class specifically: Accuracy is misleading at a roughly 39% default rate — a model that predicts "no default" for everyone still scores about 61%, and the observed accuracy is only 0.673. The gate is set on the minority class the business actually cares about. In a BFSI room, make this point explicitly; it demonstrates you understand credit risk, not just the platform.

Talk track

"Model quality is not a convention here, it is a control. This script is the promotion gate. If a retrained model drifts below the agreed ROC-AUC or F1 on the default class, the job fails and the pipeline stops. Your model risk function defines these numbers; the platform enforces them."

Stage 4b — The Same Steps as a Scheduled Pipeline (optional, 3 min)

Show this if the audience includes platform or ML engineering. It converts three interactive commands into a governed, scheduled, dependency-chained pipeline.

Job name	Script	Trigger / dependency
Generate Loan Data	<code>01_generate_data.py</code>	Manual or scheduled
Train Credit Risk Model	<code>02_train_model.py</code>	On success of Generate Loan Data

Job name	Script	Trigger / dependency
Validate Model KPIs	05_validate_model.py	On success of Train Credit Risk Model

Open Jobs in the left navigation, show the three jobs and the dependency chain, then open a past run's log. Point out that each job carries its own runtime, resource profile, schedule and email alerting, and that a failed validation job stops the chain.

Stage 5 — Deploy the Model as a Managed Endpoint

Session serving versus model deployment

	Session API (03_predict.py)	Model Deployment (cml_model.py)
Lifecycle	Lives only while the session is open	Always-on, independent of any session
Authentication	None — open inside the session	Access key required on every request
Scaling	Single Unicorn process	Configurable replicas
Entry point	Flask route POST /predict	Python function predict(args)
Port handling	Manual	Managed by Cloudera AI
Use in demo	Development-path illustration only	The production path — spend your time here

Steps

1. Go to Model Deployments in the left navigation and open "Workshop - Ravi MLOPS".
2. Walk the tabs: Overview, Deployments, Builds, Monitoring, Logs, Settings.
3. Open Settings and show the configuration a new deployment would use (see table below).
4. Open cml_model.py in the editor and show the predict(args) function — approximately fifteen lines, no serving framework, no Dockerfile.

Deployment configuration

Field	Value
Name	Workshop - Ravi MLOPS
Description	credit-risk-model Workshop - Ravi MLOPS
File	cml_model.py
Function	predict
Runtime / Kernel	Python 3.10 — Standard
CPU	2 vCPU
Memory	4 GiB
Replicas	1
Example input	See Appendix B

Do not redeploy from scratch during a live demo: A first build runs cds-sw-build.sh, which installs dependencies and retrains the model so the artifacts are baked into the image. That takes several minutes of silence. Show the existing deployment and its build history instead; if you must demonstrate a build, start it and cut away to Experiments while it runs.

How the artifacts reach the endpoint

The project is snapshotted when a model is deployed, so later development changes in the project cannot disturb a running production model — worth saying out loud, because it is the first thing an operations team worries about. `cml_model.py` loads `credit_risk_model.pkl` and `label_encoder.pkl` once at container startup, so no artifact is read per request. `cdsw-build.sh` guarantees the artifacts exist in the image by regenerating the data and retraining during the build — both scripts are seeded, so the baked model is deterministic. If you are asked how the model would instead be pulled from a registry at deploy time, that is the Cloudera AI Registry path in Section 9.1.

Talk track

"I wrote a Python function. Cloudera AI containerised it, exposed it as an authenticated REST endpoint, gave it a build history and replica control, and keeps it running without a session. There is no Dockerfile, no ingress configuration, no Kubernetes work, and no separate serving platform to buy. And the endpoint inherits the same identity and access model as the rest of the platform."

Stage 6 — Test the Endpoint

6a. From the Cloudera AI UI

1. Open the model and go to the Test tab. The example input is pre-filled.
2. Click Run and read out the response.
3. Edit the payload live: drop `credit_score` to 520, raise `debt_to_income` to 0.65, set `num_delinquencies` to 5, change `loan_purpose` to personal. Run again.
4. Contrast the two probabilities. The model should move decisively from LOW to HIGH.

Expected responses

```
// Low-risk applicant
{ "default_probability": 0.0312, "prediction": 0, "risk_label": "LOW" }

// High-risk applicant
{ "default_probability": 0.8741, "prediction": 1, "risk_label": "HIGH" }
```

Exact probabilities vary by training run. Do not quote the numbers above as guaranteed — quote the direction.

6b. From outside the platform

This is the step that convinces application teams. Retrieve the access key from Model Deployments > Workshop - Ravi MLOPS > Settings, then run from the session terminal or your laptop:

```
curl -X POST \
  https://<workbench-domain>/model?accessKey=<ACCESS_KEY> \
  -H "Content-Type: application/json" \
  -d '{
    "accessKey": "<ACCESS_KEY>",
    "request": {
      "loan_amount": 10000,
      "annual_income": 90000,
      "credit_score": 780,
      "employment_years": 10,
      "debt_to_income": 0.15,
      "num_credit_lines": 5,
      "num_delinquencies": 0,
      "loan_purpose": "home"
    }
  }'
```

Get the exact URL from the UI, not from this document: The endpoint URL format differs between Cloudera AI releases. Always copy the ready-made curl example from the model's Overview or Settings tab immediately before the demo, and test it once during pre-flight. Reading a stale URL off a slide in front of a customer is an avoidable failure.

Input validation to show if asked

- Missing fields return `{"error": "Missing required fields: [...]"}` rather than a stack trace.
- An unknown `loan_purpose` returns the list of valid values from the fitted label encoder.
- Feature order is enforced explicitly in code, so a caller cannot silently transpose columns.

Talk track

"Any application can call this — a core banking system, a loan origination workflow, a batch scoring job. It is an authenticated HTTPS endpoint that returns JSON. From the consuming team's point of view there is nothing Cloudera-specific about integrating with it."

Stage 7 — Retrain and Redeploy

Steps

1. Open `02_train_model.py` and change one hyperparameter — `max_depth` from 4 to 6 is a good choice.
2. Re-run the training script (or trigger the Train Credit Risk Model job).
3. Open Experiments and show the new run alongside the previous one — the KPI change is now on the record.
4. Run the validation script and show the gate result for the new model.
5. Go to Model Deployments > Workshop - Ravi MLOPS > Builds > Deploy New Build. Cloudera AI builds a new image with the new artifacts and cuts traffic over without downtime.
6. Show the build history: every deployment has a traceable lineage back to a build and a training run.

Timing: A rebuild takes several minutes. In a 45-minute slot, narrate steps 1–4 live, start the rebuild, and move to Stage 8 while it runs. Come back to it at the end only if it has finished.

Talk track

"Retraining is the same pipeline, not a separate project. New run, tracked. Gate re-evaluated. Rebuild, zero downtime, full build history. That is the loop your model risk policy needs — and it is the loop most organisations are doing by hand today."

Stage 8 — Automation and CI/CD

Close on automation. The project includes a CI workflow that runs on every push to the main branch of the project's Git remote:

```
Push to main
|
v
1. pip install -r requirements.txt
2. python 01_generate_data.py      -> loan_data.csv
3. python 02_train_model.py       -> credit_risk_model.pkl
4. python 05_validate_model.py    -> KPI gate
   |
   |-- ROC-AUC >= 0.72 AND F1(default) >= 0.50 -> pipeline green
   |-- either threshold missed                  -> pipeline red
```

The pipeline needs no secrets and no external services — it runs the identical scripts the SE just ran interactively. That is the point worth making: there is no separate "production version" of the code.

Extending it to trigger Cloudera AI

For a customer asking how far this goes, the next step is Cloudera AI API v2: a pipeline that passes the KPI gate can call the API to trigger a Job, create a new build and promote a deployment. Point to Section 9.3 and offer to walk their platform team through it separately — do not attempt to build it live.

Closing talk track

"Everything you have seen is one platform, one security model, one copy of the data. Development, tracking, validation, deployment, monitoring and automation. Compare that to the usual pattern: notebooks somewhere, a tracking server somewhere else, a serving stack owned by a third team, and data copied to all three. That copying is where governance breaks — and it is what Cloudera removes."

8. Compressed Version (10 minutes)

For a booth, an executive drop-in, or a slot that has been cut in half:

1. Open the project, walk the Files tab as "the lifecycle in eight files" (1 min).
2. Experiments: sort by roc_auc, compare two runs, open the artifacts of the best one (4 min).
3. Show 05_validate_model.py thresholds and state that this is the promotion gate (1 min).
4. Models: show the green endpoint, run a low-risk and a high-risk prediction from the Test tab (3 min).
5. Close with the one-platform message (1 min).

Skip live training, skip curl, skip the rebuild. Never start a build you cannot finish inside the slot.

9. Optional Extensions

Validate before you demo: The extensions below are not part of the scripted flow in the MLOPS Workshop project. They depend on services being enabled in the demo environment. Test each one yourself before showing it to a customer, and describe rather than click if you have not.

9.1 Cloudera AI Registry

The demo deploys from project artifacts. The governed path is to register the model from its Experiment run into Cloudera AI Registry, then deploy from the registry. The training script already logs the model with a signature and input example, which is the prerequisite for registration.

- From Experiments, open the best run, click Register Model, and give it a name, description and version notes.
- The registry stores the model with its version, parameters, metrics, artifacts, metadata and lineage — who created it, when, and from which run.
- Registered versions can then be deployed to a project or to a Cloudera AI Inference endpoint, and access is controlled by ownership and visibility settings.

Use this when the conversation turns to model governance, audit or model risk management — for a BFSI audience, registry and lineage often matter more than anything in Stages 5 to 7.

9.2 Cloudera AI Inference

Cloudera AI Inference is the production-grade serving path for both predictive and generative models, exposing endpoints over the Open Inference Protocol (predictive) or the OpenAI API (generative), with autoscaling, canary deployment and Prometheus metrics. The Workbench Model Deployment used in this demo is the fast, project-scoped path. Be precise about the difference — a customer sizing a production platform needs to know which one they are buying into.

9.3 Deployment paths and API v2 promotion

There are three routes to a live endpoint: directly from the UI (what this demo uses), from a version registered in Cloudera AI Registry, or from a CI/CD pipeline calling the SDK. For offline scoring the same model runs as a batch Job on a schedule, is invoked through the SDK, or is orchestrated from Airflow in Cloudera Data Engineering — one trained model, several consumption patterns, no rewrite between them.

Cloudera AI exposes a REST API and a Python client for projects, jobs, models and builds. A CI pipeline that has passed the KPI gate can use it to trigger a training job, create a build and promote a deployment without a human in the UI. This is the natural follow-up conversation after Stage 8, and a strong reason to bring in a platform architect for a second session.

9.4 Model monitoring and the closed loop

Be precise here, because the common shorthand — "Cloudera has no drift detection" — understates the platform and will be contradicted by anyone who has used it.

- **Native.** A model metrics store, written to from the deployed model with `track_metric()`, and read back with `read_metrics()`. Each prediction is individually identifiable, down to feature level.
- **Native.** Ground-truth reconciliation — `track_delayed_metric()` and `track_aggregate_metrics()` write the real outcome back against the original prediction once it is known, which is how model decay becomes measurable.
- **Native.** Non-functional endpoint monitoring — success rate, requests per second and request counts per replica, on the model's Monitoring tab.
- **Yours to write.** The statistical drift test itself — typically a significance test comparing a reference window against the current one. The Continuous Model Monitoring AMP, built on open-source Evidently AI, is the usual starting point, and Cloudera AI Applications can host the resulting dashboard.

The closed loop customers build on top of this is: serve, record, reconcile against ground truth, detect decay, retrain through the same job pipeline, re-apply the KPI gate, promote the new build. Every step except the drift test is platform functionality.

9.5 Serving the model through an Application

`03_predict.py` runs a Flask API under Gunicorn and resolves its port from `CDSW_APP_PORT`, so it can also be published as a Cloudera AI Application to give a long-running, shareable URL. Useful when a customer wants to see a model behind a UI rather than behind a REST call, but it duplicates Stage 5 — pick one, do not do both.

10. Reset and Shared-Environment Etiquette

Leave the project ready for the next SE. This takes two minutes.

Action	Why
Stop your session	Sessions hold quota and block the next SE's session start
Revert any code edits made during Stage 7	Hyperparameter changes left in place skew the next demo's Experiments view
Leave the deployed model running and green	Rebuilding it costs the next SE several minutes of pre-flight
Do not delete Experiment runs	The comparison view needs run history to be worth showing
Do not push to the project's Git remote	The shared demo project should stay at a known-good state
Stop any Application you started	Applications run until explicitly stopped and consume quota continuously

If you need to change the demo: Create your own project from the same Git remote — Project Settings > Git shows it — and modify your copy. Never modify the shared project to suit one customer conversation.

11. Questions You Will Be Asked

Question	How to answer
"Is this just MLflow?"	Yes, the tracking API is open-source MLflow, deliberately. The value is that it is managed, secured under SDX, tied to project permissions, and connected to registry and deployment. No proprietary API means no migration cost if they leave.
"The data is synthetic — would this work on our real data?"	The synthetic generator is only step one. Replace it with a Spark or Hive read, or an Iceberg table through a Data Connection; every downstream stage is unchanged. Offer to swap in a sample of their schema for a follow-up session.
"How do we stop a bad model reaching production?"	Point back to Stage 4. The gate is code, the thresholds are owned by model risk, and the non-zero exit code stops the job and the pipeline. Add registry-based promotion (Section 9.1) for a formal approval step.
"Can we use PyTorch / TensorFlow / scikit-learn / R instead?"	Yes. The demo uses XGBoost, but the pattern — session, tracked run, artifact, predict function, endpoint — is framework-agnostic. Runtimes are selectable and customisable, including GPU editions.
"What about drift?"	Split the answer. The metrics store, per-prediction tracking and ground-truth reconciliation are native, and so is the retraining pipeline. The statistical drift test is code you own, usually started from the Continuous Model Monitoring AMP built on Evidently AI. Answering it this way is both accurate and stronger than a flat no.
"How does this compare to SageMaker / Vertex / Databricks?"	Stay on the ground you own: data gravity and governance. The lifecycle runs where the data already sits, under one security and lineage model, on-premises, in cloud, or hybrid — with the same experience. Avoid feature-by-feature comparison unless you have current, verified detail.
"How many replicas can we run? What is the latency?"	Replicas are configurable per deployment; do not quote latency numbers from this demo — a single-replica deployment scoring one row is not a benchmark. Offer a properly scoped sizing conversation instead.

Question	How to answer
"Can our data scientists work in Jupyter?"	Yes. Editor choice is per session — Workbench, JupyterLab, or a bring-your-own editor runtime. Nothing in this demo depends on the Workbench editor.

Appendix A — Features and Model

Feature	Description	Type
loan_amount	Requested loan amount	Integer
annual_income	Applicant annual income	Integer
credit_score	FICO-style score, 300–850	Integer
employment_years	Years in current employment	Integer
debt_to_income	Monthly debt payments over gross income	Float, 0.05–0.75
num_credit_lines	Open credit lines	Integer
num_delinquencies	Past delinquencies	Integer
loan_purpose	home, auto, education, personal, business	Categorical, label-encoded

Model configuration

Setting	Value
Algorithm	XGBClassifier (XGBoost 1.7.6)
n_estimators	100
max_depth	4
learning_rate	0.1
eval_metric	logloss
random_state	42
Train/test split	80/20, stratified on the default label
Target	default — 1 indicates default, 0 indicates no default
Approximate default rate	About 39% of records

Appendix B — Sample Payloads

Low-risk applicant

```
{
  "loan_amount": 10000,
  "annual_income": 90000,
  "credit_score": 780,
  "employment_years": 10,
  "debt_to_income": 0.15,
  "num_credit_lines": 5,
  "num_delinquencies": 0,
  "loan_purpose": "home"
}
```

High-risk applicant

```
{
  "loan_amount": 25000,
  "annual_income": 22000,
  "credit_score": 520,
  "employment_years": 1,
  "debt_to_income": 0.65,
  "num_credit_lines": 2,
  "num_delinquencies": 5,
  "loan_purpose": "personal"
}
```

Endpoint response shape

```
{
  "success": true,
  "response": {
    "default_probability": 0.0312,
    "prediction": 0,
    "risk_label": "LOW"
  }
}
```

Deliberate demo choice: Change exactly one field at a time when improvising in front of a customer — credit_score alone, then debt_to_income alone. Changing several at once makes the model look arbitrary; changing one makes it look explainable.

Appendix C — Claim Discipline

Two lists worth reading before your first delivery.

Safe to claim

- Experiment tracking is native, automatic, and requires no tracking-server configuration in the code.
- Models deploy from a plain Python function — no Dockerfile, no Kubernetes work by the user.
- Endpoints are authenticated and inherit the platform's identity and access model.
- Runtimes, resource profiles and replicas are configurable per workload.
- The same code runs in a session, a job and a CI pipeline unchanged.
- Rebuild-and-redeploy is zero downtime for the endpoint.

Do not claim

- That the statistical drift test is native — the metrics store, ground-truth tracking and retraining pipeline are, but the drift test itself is customer code, usually started from the Evidently AI accelerator.
 - Any latency or throughput figure based on this demo. It is a single-replica, single-row-inference deployment.
 - That the tracking layer is proprietary Cloudera technology. It is MLflow, and a technical audience will know.
 - Specific feature parity against SageMaker, Vertex or Databricks without verifying current detail first.
 - That model accuracy figures from synthetic data indicate anything about performance on real loan data.
-

Feedback and improvements to this asset are welcome — it is intended to be maintained by the SE community that uses it.